```
class BookInStock
end
```

As we saw in the previous chapter, we can create new instances of this class using new:

```
a_book = BookInStock.new
another_book = BookInStock.new
```

After this code runs, we'd have two distinct objects, both of class BookInStock. But there's nothing to distinguish one instance from the other, aside from the fact that they have different internal object ids. Worse, these objects don't yet hold any of the information we need them to hold.

The best way to fix this is to provide the class with an initialize method. This method lets us set the state of each object as it is constructed. We store this state in *instance variables* inside the object. (Remember instance variables? They're the ones that start with an @ sign.) Because each object in Ruby has its own distinct set of instance variables, each object can have its own unique state.

So, here's our updated class definition:

```
class BookInStock
  def initialize(isbn, price)
    @isbn = isbn
    @price = Float(price)
  end
end
```

The initialize method is special in Ruby programs. When you call BookInStock.new to create a new object, Ruby allocates some memory to hold an uninitialized object and then calls that object's initialize method, passing through all arguments that were passed to new. This gives you a chance to write code that sets up your object's state.

For class BookInStock, the initialize method takes two parameters. These parameters act just like local variables within the method, so they follow the local variable naming convention of starting with a lowercase letter. But, as local variables, they would just evaporate once the initialize method returns, so we need to transfer them into instance variables. This is very common behavior in an initialize method—the intent is to have our object set up and usable by the time initialize returns.

This method also illustrates something that often trips up newcomers to Ruby. Notice how we say @isbn = isbn. It's easy to imagine that the two variables here, @isbn and isbn, are somehow related. It looks like they have the same name, but they don't. The former is an instance variable, and the "at" sign is actually part of its name.

Finally, this code illustrates a simple piece of validation. The Float method takes its argument and converts it to a floating-point number, terminating the program with an error if that conversion fails. Later in the book we'll see other, more resilient, ways to handle these exceptional situations. (We know that we shouldn't be holding prices in inexact old floats. Ruby has classes that hold fixed-point values exactly, but we want to look at classes, not arithmetic, in this section.)

What we're doing here is saying that we want to accept any object for the price parameter as long as that parameter can be converted to a float. We can pass in a float, an integer, or even a string containing the representation of a float, and it will work. Let's try this now. We'll